

OSAWARE — Wallet Integration

A specification for browser-extension wallet support in OSAWARE.

1. Scope

Stage 1 adds read-only crypto-wallet support to OSAWARE via browser-extension wallets (MetaMask, Rabby, Coinbase Wallet extension, Frame, Brave Wallet — any wallet that implements **EIP-1193** and announces itself via **EIP-6963**).

The integration runs entirely in the browser. There is no backend dependency, no third-party SDK, no API key, no relay server, no QR code, no mobile-wallet pairing.

What this delivers

- A new `WALLETCONNECT` flow that opens the user's wallet extension, lets them approve the OSAWARE origin, and binds the resulting address to the current BASIC program's process memory.
- Read-only access from BASIC to: the connected address, the network name and chain ID, the user's native ETH balance, and the user's balance in a curated list of ERC-20 tokens.
- A `WALLETCONNECT.BAS` test program that exercises the full flow.

Stage 2 (not in this spec, but designed-for)

- Server-side proof-of-ownership via SIWE (Sign-In with Ethereum, EIP-4361).
- Persistent binding of a wallet address to an OSAWARE user account in the backend.
- Admin-panel visibility of connected wallets.

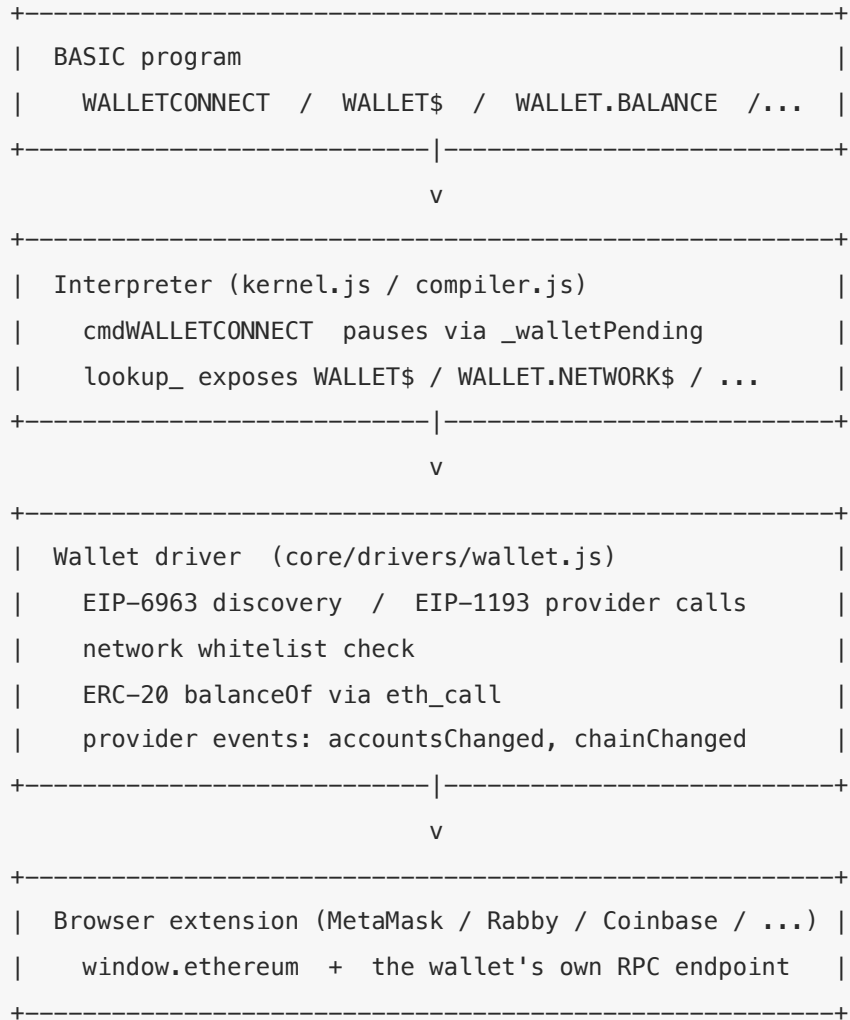
Stage 1 binds wallet-to-program-process; Stage 2 binds wallet-to-user-account.

Explicitly left out

- **WalletConnect v2 / mobile wallets / QR code flow.** Only browser extensions.
- **Any signing or transaction-sending command.** Read-only.
- **Hardware wallet UX flourishes.** Hardware wallets behind MetaMask still work transparently.
- **NFT enumeration.** Requires paid indexer APIs; deferred.

- **L2 / sidechain support.** Whitelist starts at Ethereum mainnet only and is one config object away from adding Polygon / Base / Arbitrum etc.
- **Token swaps, price quotes, DeFi reads.** Out of scope for v1.

2. Architecture



The driver mirrors the GL driver pattern (`gfx.js`, `gl3d.js`) — a separate JS module loaded by `index.html`, mixed onto the interpreter, gated by a pending-flag for async work.

3. Wallet discovery — EIP-6963

The old single-`window.ethereum` pattern races when the user has multiple wallet extensions installed. EIP-6963 fixes this: each extension announces itself.

```
// Driver init:
this._walletProviders = [];
window.addEventListener('eip6963:announceProvider', (e) => {
  this._walletProviders.push(e.detail); // {info: {name, icon, rdns, uuid}, provider
});
window.dispatchEvent(new Event('eip6963:requestProvider'));
```

All installed extensions answer synchronously within the same event-loop tick. The driver keeps the list keyed by `info.rdns` (reverse-DNS unique ID).

Picker UX

- **Zero installed wallets** — `WALLETCONNECT` prints `No wallet extension detected`. Install `MetaMask`, `Rabby`, or `Coinbase Wallet`. and returns immediately.
- **Exactly one wallet** — auto-selected, no picker shown.
- **Multiple wallets** — small terminal picker: `` Pick a wallet:
 1. MetaMask
 2. Rabby
 3. Coinbase Wallet

`` User types the number, picker closes, flow continues.

Fallback to `window.ethereum`

If EIP-6963 returns nothing but `window.ethereum` exists (rare with modern wallets, but possible with old setups), the driver synthesises a `{info: {name: 'Browser Wallet', rdns: 'legacy'}, provider: window.ethereum}` entry.

4. Connection flow

```
WALLETCONNECT issued
-> driver: pick provider (auto or via picker)
-> provider.request({method: 'eth_requestAccounts'})
```

```

-> extension popup
-> user approves
-> returns ['0xABC...']
-> provider.request({method: 'eth_chainId'}) -> '0x1'
-> network whitelist check
    -> 0x1 OK / anything else => reject with NETWORK NOT SUPPORTED
-> bind address to interpreter's process memory:
    interpreter._walletAddress = '0xABC...'
    interpreter._walletChainId = 1
    interpreter._walletProvider = providerRecord
    interpreter._walletPending = false
-> subscribe provider.on('accountsChanged') / .on('chainChanged')
-> resume BASIC program

```

During the wait, `tick()` checks `_walletPending` (added to the existing guard alongside `_glLoadPending`) and yields. The user can cancel from the wallet UI; on rejection the driver reports `User rejected connection.` and continues with no binding.

Subsequent connect calls

If the program issues `WALLETCONNECT` while already connected, the driver no-ops and returns the existing address. If the extension was disconnected externally (user revoked permission in the wallet), the driver detects it via `eth_accounts` returning `[]` and re-triggers the request flow.

5. Network whitelist

```

// core/drivers/wallet.js - top of file
const SUPPORTED_NETWORKS = {
  1: { name: 'Ethereum', symbol: 'ETH', explorer: 'https://etherscan.io' },
  // Future: 137 Polygon, 8453 Base, 42161 Arbitrum, 10 Optimism
};

```

Stage 1 ships with chain ID 1 only. Adding a chain is **one line** plus a token-registry entry per token-on-chain.

If the user's wallet is on an unsupported chain, `WALLETCONNECT` prints:

Wallet is on Polygon (chain 137). OSAWARE supports: Ethereum.
Switch network in your wallet, then re-run WALLETCONNECT.

The driver listens for `chainChanged` events and, if the user switches to a supported chain after connecting, transparently rebinds. If they switch to an unsupported chain, the driver clears the binding (`WALLET$` becomes `""`) and prints a one-line notice.

6. BASIC API surface (read-only v1)

Statements (effects)

<code>WALLETCONNECT</code>	' run the discovery + connect flow; blocks until done
<code>WALLETDISCONNECT</code>	' clear local binding; does NOT revoke extension permission

Variables / expressions (reads)

<code>WALLET\$</code>	' "0xABC..." or "" if not connected
<code>WALLET.NETWORK\$</code>	' "Ethereum" / "" if not connected
<code>WALLET.CHAINID</code>	' 1 / 0 if not connected
<code>WALLET.BALANCE</code>	' float, native token in standard units (ETH on mainnet)
<code>WALLET.CONNECTED</code>	' 1 if connected, 0 otherwise

Token reads

<code>WALLET.TOKEN(symbol\$)</code>	' balance as float, e.g. <code>WALLET.TOKEN("USDC")</code>
<code>WALLET.TOKENS\$</code>	' comma-joined "USDC=1240.55,DAI=10.2,..." across the registry

`WALLET.TOKEN("BAD")` returns `0` for unknown symbols. `WALLET.TOKENS$` returns only the tokens with non-zero balances.

Initial token registry (Ethereum mainnet)

Symbol	Contract address	Decimals
USDC	0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48	6

Symbol	Contract address	Decimals
USDT	0xdAC17F958D2ee523a2206206994597C13D831ec7	6
DAI	0x6B175474E89094C44Da98b954EedeAC495271d0F	18
WETH	0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2	18
WBTC	0x2260FAC5E5542a773Aa44fBCfeDf7C193bc2C599	8
LINK	0x514910771AF9Ca656af840dff83E8264EcF986CA	18
UNI	0x1f9840a85d5aF5bf1D1762F925BdAdDC4201F984	18

The registry is a const at the top of `wallet.js`. Adding a token is one line.

Read mechanism: standard ERC-20 `balanceOf(address)` selector `0x70a08231` followed by 32-byte address pad, sent as an `eth_call` via the user's provider. Decimals are baked into the registry, not fetched (avoids a per-token round trip).

7. Kernel integration

New process-memory fields

Added to `ProcessMemory.reset()`:

```
this._walletAddress = '';
this._walletChainId = 0;
this._walletProvider = null;    // { info, provider } reference
this._walletPending = false;    // async-pause flag
this._walletEvents = null;      // listener cleanup handle
this._walletTokenCache = new Map(); // {symbol -> {value, ts}} short TTL
```

Wallet state is **process-local**. A second BASIC program starts with no connection, even though the extension itself may still trust the OSAWARE origin (one-click reconnect when it calls `WALLETCONNECT`).

Tick guard

`tick()` already short-circuits on `_glLoadPending`. Add `_walletPending`:

```
if (this.want_keypress || this.want_input || this._glLoadPending ||
    this._walletPending || this._resizePending) return;
```

Driver mixin

`core/drivers/wallet.js` exports a `WalletDriver` class whose methods get mixed onto the Interpreter prototype, same pattern as `GfxDriver` / `Gl3dDriver`. No changes to the kernel's process/scheduler model.

8. Error model

Errors are reported via `appendLine(msg, 1)` and the BASIC program continues. Read-only operations never throw to the interpreter level.

Condition	BASIC-visible behaviour
No extension installed	<code>WALLETCONNECT</code> prints <code>No wallet extension detected.</code> and returns; <code>WALLET\$</code> stays <code>""</code> .
User rejects connection	<code>WALLETCONNECT</code> prints <code>User rejected connection.</code> and returns; <code>WALLET\$</code> stays <code>""</code> .
Wallet on unsupported chain	<code>WALLETCONNECT</code> prints <code>Wallet is on <name> (chain N).</code> <code>OSAWARE supports: Ethereum.</code> and returns.
Wallet switches to unsupported chain mid-program	Binding clears; <code>WALLET\$</code> becomes <code>""</code> ; <code>Wallet switched to unsupported network.</code> printed.
Wallet switches account mid-program	Binding updates silently; <code>WALLET\$</code> reflects new address.
RPC failure on balance read	Returns <code>0</code> for that read; no print (avoids spamming a tight loop).
Unknown token symbol passed to <code>WALLET.TOKEN</code>	Returns <code>0</code> . No print.
<code>WALLETDISCONNECT</code> while not connected	No-op. No print.

9. Security

1. **HTTPS only.** Already enforced by nginx on osaware.com. Extensions refuse to inject on insecure origins anyway.
 2. **No `eth_sendTransaction`, `eth_signTransaction`, `eth_signTypedData_*`, or `personal_sign`** is exposed in Stage 1. The driver method that builds the provider call refuses any method name outside an allowlist:
 3. `eth_requestAccounts`
 4. `eth_accounts`
 5. `eth_chainId`
 6. `eth_getBalance`
 7. `eth_call`
 8. **Domain isolation is automatic.** Browser extensions scope permissions per origin.
 9. **No private keys ever touch OSAWARE memory or storage.** The driver never even sees private keys; the wallet handles signing internally.
 10. **Stage 1 binding is unverified.** A user signing in their browser doesn't prove anything to a server. Stage 2 SIWE fixes this. Don't gate server features on Stage 1 binding.
 11. **Token registry is fixed in code.** Adding tokens requires a code change + redeploy. Prevents a hostile program from declaring a fake "USDC" pointing at a malicious contract.
 12. **WALLETDISCONNECT clears local state only.** The extension itself still trusts the origin until the user revokes that permission in the wallet UI. This matches user expectation across the rest of the dapp world.
-

10. The WALLETCONNECT test program

Embedded in `vfs.js` under the `MISC` or new `WALLET` category. Demonstrates every command and serves as the basic happy-path smoke test.

```
10 REM WALLETCONNECT - smoke test for the wallet driver
20 CLS : COLOUR 3
30 PRINT "+-----+"
40 PRINT "|   OSAWARE WALLET CONNECT   |"
50 PRINT "+-----+"
60 COLOUR 7 : PRINT ""
70 PRINT "Press any key to begin..."
80 DUMMY = GETKEY()
90 PRINT "Opening your wallet extension..."
```



```

100 WALLETCONNECT
110 IF WALLET$ = "" THEN PRINT "" : PRINT "  No wallet connected." : END
120 PRINT ""
130 PRINT "  Address : "; WALLET$
140 PRINT "  Network : "; WALLET.NETWORK$
150 PRINT "  Chain ID: "; WALLET.CHAINID
160 PRINT "  Native  : "; WALLET.BALANCE; " ETH"
170 PRINT ""
180 PRINT "Token balances (non-zero only):"
190 PRINT "  "; WALLET.TOKENS$
200 PRINT ""
210 PRINT "Press D to disconnect, any other key to exit."
220 K = GETKEY()
230 IF K = 68 OR K = 100 THEN WALLETDISCONNECT : PRINT "Disconnected." : END
240 END

```

Add a new DIR category `WALLET` (or place under `MISC`), with `WALLETCONNECT` as its first entry.

11. Implementation plan

Files to create

- `core/drivers/wallet.js` — driver (~300 lines).

Files to modify

- `core/kernel.js` — register driver, add `_walletPending` to tick guard, add command-table entries for `WALLETCONNECT` / `WALLETDISCONNECT`, add built-ins for `WALLET$` / `WALLET.NETWORK$` / `WALLET.CHAINID` / `WALLET.BALANCE` / `WALLET.CONNECTED`.
- `core/compiler.js` — wire `WALLET$`, `WALLET.NETWORK$`, `WALLET.CHAINID`, `WALLET.BALANCE`, `WALLET.CONNECTED` into `lookup_`. Wire `WALLET.TOKEN(symbol$)` and `WALLET.TOKENS$` into the function-call paths and `_BUILTIN_NUM_NAMES` / volatile sets so they aren't cached.
- `core/kernel/process_memory.js` — add the six new fields to `reset()`.
- `core/vfs.js` — add the `WALLETCONNECT` program; add it to a `WALLET` category in the DIR listing.

- `index.html` — load `core/drivers/wallet.js` after `kernel.js` and before `boot.js`, with a cachebuster.

Cachebuster

Bumped via `./bump.sh` as always. Single new `<script>` reference in `index.html` increases the count from 22 to 23.

Verification

1. `WALLETCONNECT` with no extension installed → prints the expected error, no crash.
2. `WALLETCONNECT` with MetaMask on Goerli/Sepolia → prints "Wallet is on Goerli (chain 5). OSAWARE supports: Ethereum."
3. `WALLETCONNECT` with MetaMask on mainnet → opens extension, shows address, displays balance.
4. `WALLET.TOKEN("USDC")` with a known USDC holder address → returns correct balance.
5. `WALLET.TOKEN("BOGUS")` → returns 0, no error.
6. Switch wallet account while program running → `WALLET$` updates.
7. Switch network to Polygon while program running → `WALLET$` clears, notice printed.
8. `WALLETDISCONNECT` → `WALLET$` clears.
9. Second program: starts disconnected, can `WALLETCONNECT` independently.
10. `TESTS00P` and `TESTS` still pass (the wallet additions don't touch hot paths in `compiler.js`).

12. Stage 2 — sketch (not in this spec)

When Stage 2 lands:

- A new `WALLETPROVE BASIC` command initiates the SIWE handshake. Backend issues a nonce; driver builds the SIWE message with `osaware.com` as domain and the connected chain ID; user signs via `personal_sign`; backend verifies via `viem.verifyMessage` and binds the address to the logged-in user's account.
- `personal_sign` is added to the provider-call allowlist **only** for Stage 2 and **only** within the `WALLETPROVE` flow. The general allowlist stays read-only.
- New SQLite columns on `users`: `wallet_address TEXT NULL`, `wallet_chain_id INTEGER NULL`, `wallet_proved_at INTEGER NULL`.
- Admin panel surfaces these columns.

- Stage 2 is additive — Stage 1 commands keep their browser-local semantics. `WALLETPROVE` is the only Stage-2-only command.

This spec describes Stage 1 of OSAWARE's wallet integration. It is intentionally narrow: browser-extension wallets only, Ethereum mainnet only, read-only. It is the smallest change that delivers a useful, secure surface and leaves a clean path to Stage 2 server-verified binding.